Cross Toolchain for Raspberry Pi

HPC Laboratory

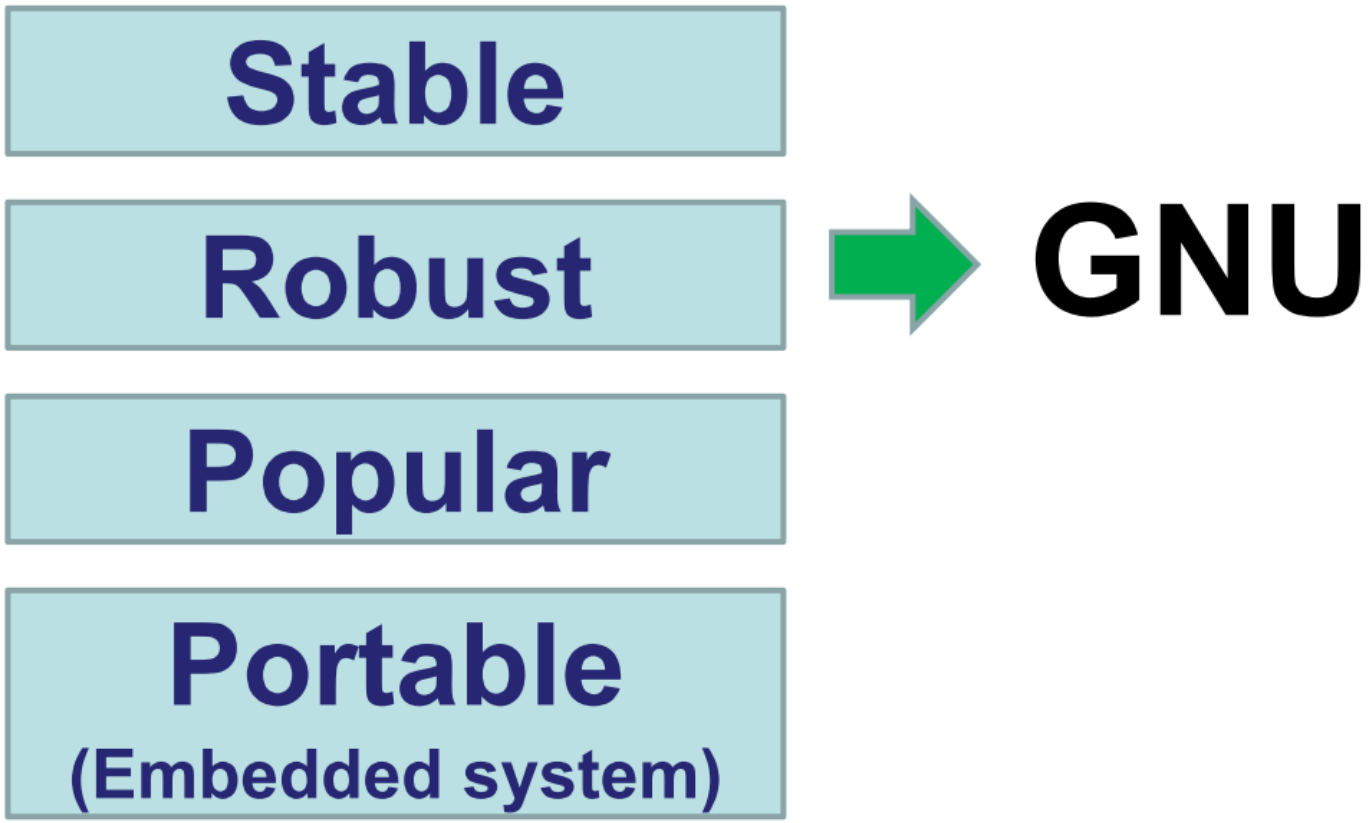
What can You Learn?

- 由 source code開始，自己編譯支援 Raspberry Pi相關的cross compiler、cross assembler、cross linker等工具

Introduction

- **A total solution for embedded system development**
  - Hardware => SOC, MCU, peripheral…
  - Software => software-development tools, applications
- **Software-development tools**
  - Compiler, assembler, linker, debugger, C standard library, IDE, …
  - Commercial => Very high cost
  - Open source => Free. Users can modify, distribute, and use the software

Software-Development Tools (Open Source)



GNU Binutils

- The GNU binutils is a collection of binary tools
  - ld - the GNU linker
  - as - the GNU assembler
  - Other binary tools
    - ar - A utility for creating, modifying and extracting from archives
    - gprof - Displays profiling information
    - objcopy - Copys and translates object files
    - objdump - Displays information from object files (disassembler)
    - …
- Easy to port binutils to other platforms
- Suitable for embedded-system development

GNU C/C++ Compiler



- Retargetable
- Available from the Free Software Foundation
  - GPL Licensed
  - Free
- Written by *Richard Stallman* originally (FSF)
- Front-end, back-end system with support for many of each
- Code quality is better than some known compiler, very close to DEC's compiler on Alpha
- Supplied as vendor compiler by NeXT, BeOS, Linux, BSD

Front Ends

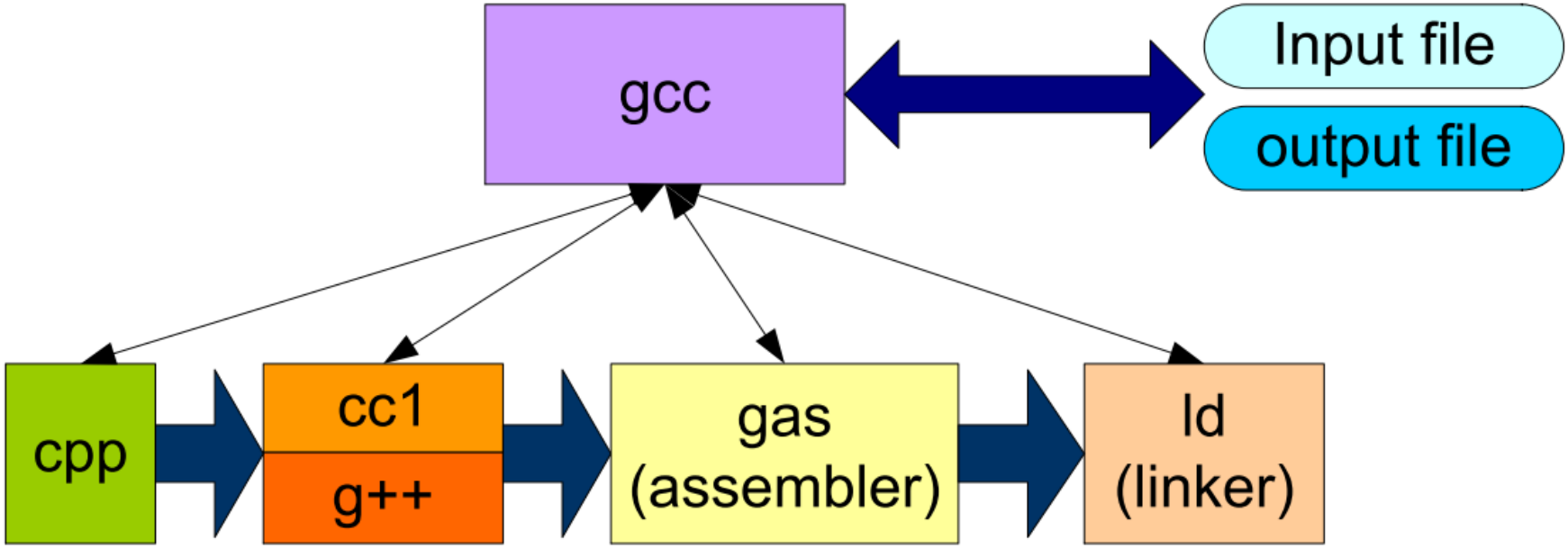
- C、C++、Objective C
- Ada 95 (GNAT)
- Fortran77、Fortran95
- Pascal
- Modula-2、Modula-3
- Java (supported from gcc 3.0)
  - Java->Bytecode, Bytecode->native code
  - Java->Native code
- Cobol
- Chill (Cygnus, a language in the Modula II family used in European telecommunications)

Machines Supported by GCC

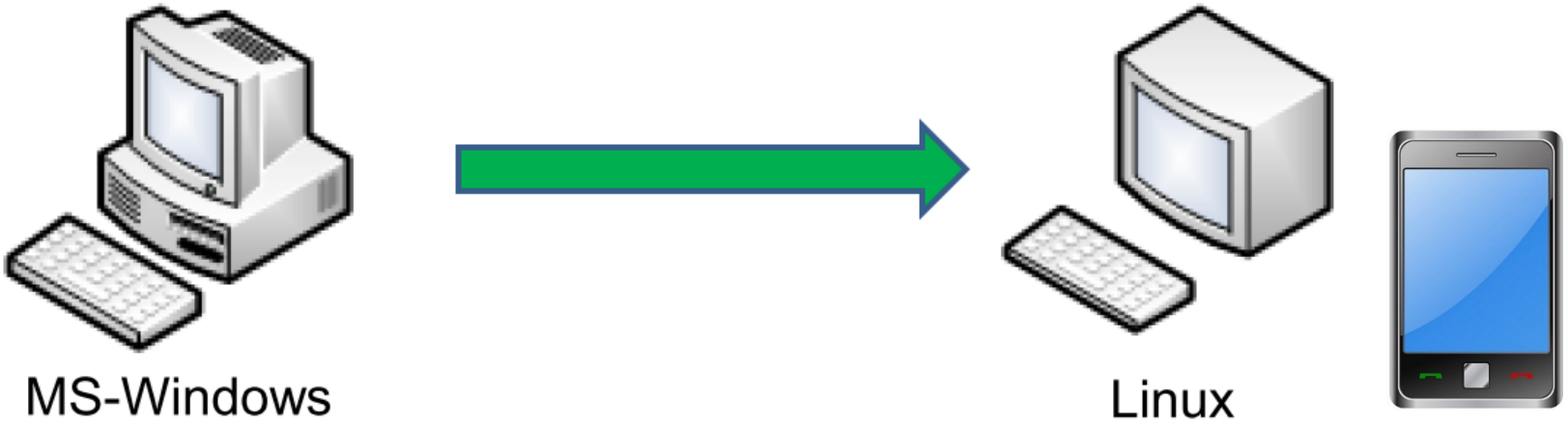
- Essentially all machines in widespread.
- ARM
- Alpha (DEC)
- Intel x86 Families、i860、i960
- Motorola 680x0、68C11、DSP56000
- Hitachi SH、H8300
- MIPS、IBM PowerPC、HP PA-RISC、SUN SPARC
- Intel IA64, AMD x86-64
- Cell processor (used by PS3)
- …

More than 30 architectures

Compilation



- Q1:
  - Build a cross compiler which executes on MS-Windows and produces Linux executable code



Target machine

Machine Classification

- **Build machine**
  - The machine which builds cross-toolchains
- **Host machine**
  - The machine which cross-toolchains will execute on
- **Target machine**
  - The machine which cross-toolchains will produce output for

一個完整的**compiler**，包含了:

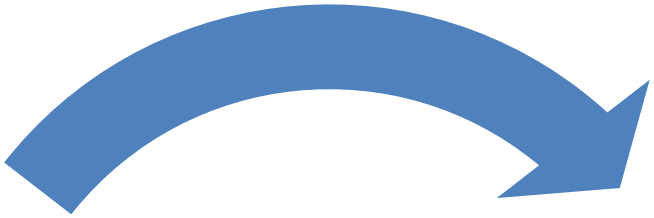
Cross Compiler

- Compiler
- Runtime library

  Handle operations that are not directly supported by the hardware, the OS, or code generation.

  For example: division operations, I/O functions, malloc(), …

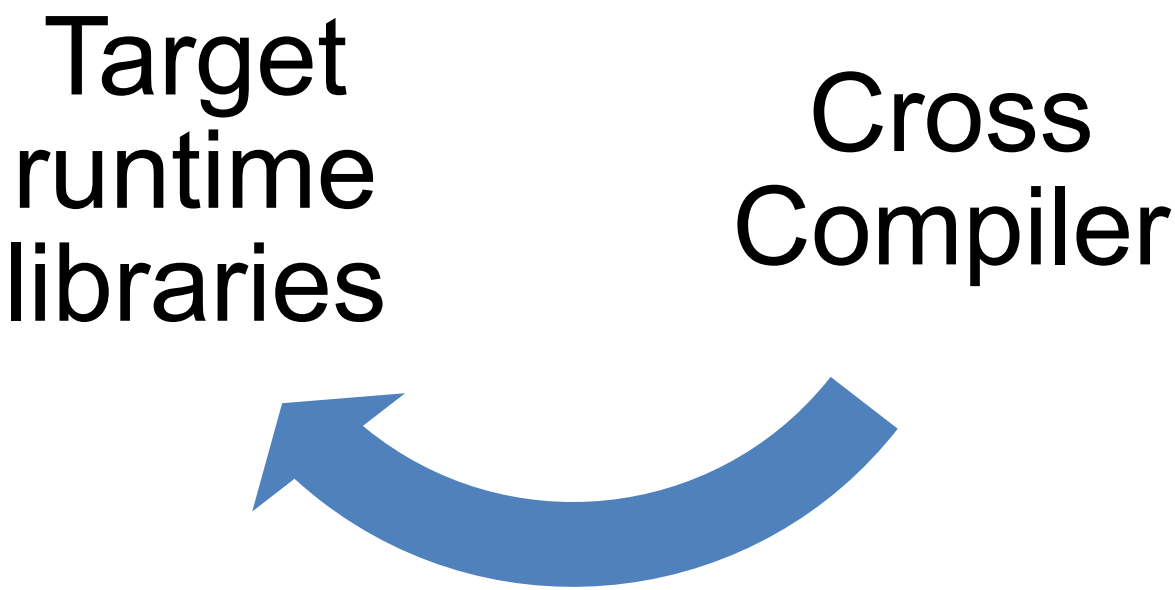
Cross Compiler



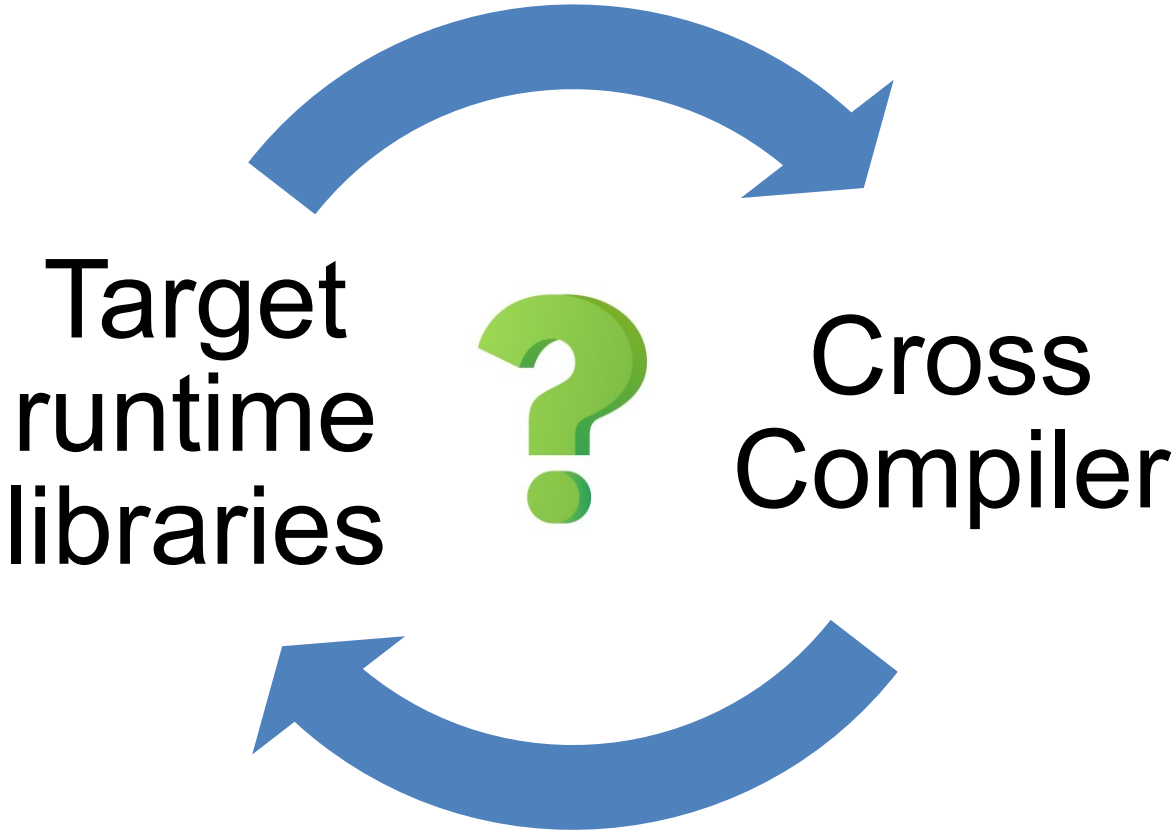
Target runtime libraries

Cross Compiler

Cross Compiler



Cross Compiler



Cross Compiler



實驗步驟

實驗步驟 (1)

- 在PC上安裝Linux作業系統
  - Ex: Ubuntu (64-bit version) based on VMware or Virtual PC.
  - 或其他Linux distribution

實驗步驟 (2)

- **Assume:**
  - --target=arm-linux-gnueabihf
- **下載GCC 12.3.0、binutils-2.40、GLIBC 2.38、Linux kernel**
- Build cross binutils (bare-metal cross compiler use)
- Build a bare-metal cross compiler for ARM
- **Build target header files and runtime libraries**
  - Build a GLIBC
  - Or copy from the target machine
- **Build cross binutils**
- **Build a cross compiler for ARM Linux**

Download Source Code

- **Binutils 2.40**
- **GCC 12.3.0**
- **GLIBC 2.38**
- **Linux kernel (from Raspberry Pi website)**

你可以嘗試使用較新的版本，建構cross toolchain。

Download Source Code: binutils

- http://ftp.gnu.org/gnu/binutils/

  Download binutils-2.40.tar.gz

Download Source Code: GCC

- https://ftp.gnu.org/gnu/gcc/

  Download gcc-12.3.0.tar.gz

Download Source Code: Linux kernel

```
$ git clone \
https://github.com/raspberrypi/l
inux --depth 1
```

Download Source Code: glibc

- https://ftp.gnu.org/gnu/glibc/

  Download glibc-2.38.tar.gz

Build cross toolchain from source codes

1. Build Cross Binutils

- $ mkdir WORK　　(WORK目錄的位置自己設定)
- $ cd WORK
- $ tar -zxvf binutils-2.40.tar.gz
- $ mkdir build_binutils
- $ cd build_binutils
- $ ../binutils-2.40/configure \
  --prefix=**/home/pschen/**WORK/crossgcc1\
  --target=arm-linux-gnueabihf
- $ make
- $ make install

/home/pschen/請改為你自己的home directory
後面的投影片也一樣

2. Build a Bare-metal Cross Compiler (1)

- **安裝GMP、MPFR、MPC**
  - 使用apt-get install libgmp-dev libmpfr-dev libmpc-dev

- $ cd WORK
- $ tar -zxvf gcc-12.3.0.tar.gz
- $ mkdir build_gcc1
- $ cd build_gcc1

2. Build a Bare-metal Cross Compiler (2)

- Add "/home/pschen/WORK/crossgcc1/bin" to PATH
- $ ../gcc-12.3.0/configure \
  --prefix=/home/pschen/WORK/crossgcc1 \
  --target=arm-linux-gnueabihf \
  --enable-languages=c **--without-headers** \
  --disable-libmudflap --disable-libatomic \
  --with-arch=armv6 --disable-shared \
  --enable-static --disable-decimal-float \
  --disable-libgomp --disable-libitm \
  --disable-libquadmath --disable-libsanitizer \
  --disable-libssp **--disable-threads** \
  --with-float=hard --with-fpu=vfp
- $ make
- $ make install

Cross Compiler

OK.
In /home/pschen/WORK/crossgcc1



3-1. Installing Kernel Headers

Change to the directory of Linux kernel:

- `$ cd linux`
- `$ make headers_install` **`ARCH=arm`** `\`
  `INSTALL_HDR_PATH=`**`/home/pschen/WORK/sysroot/usr/`**

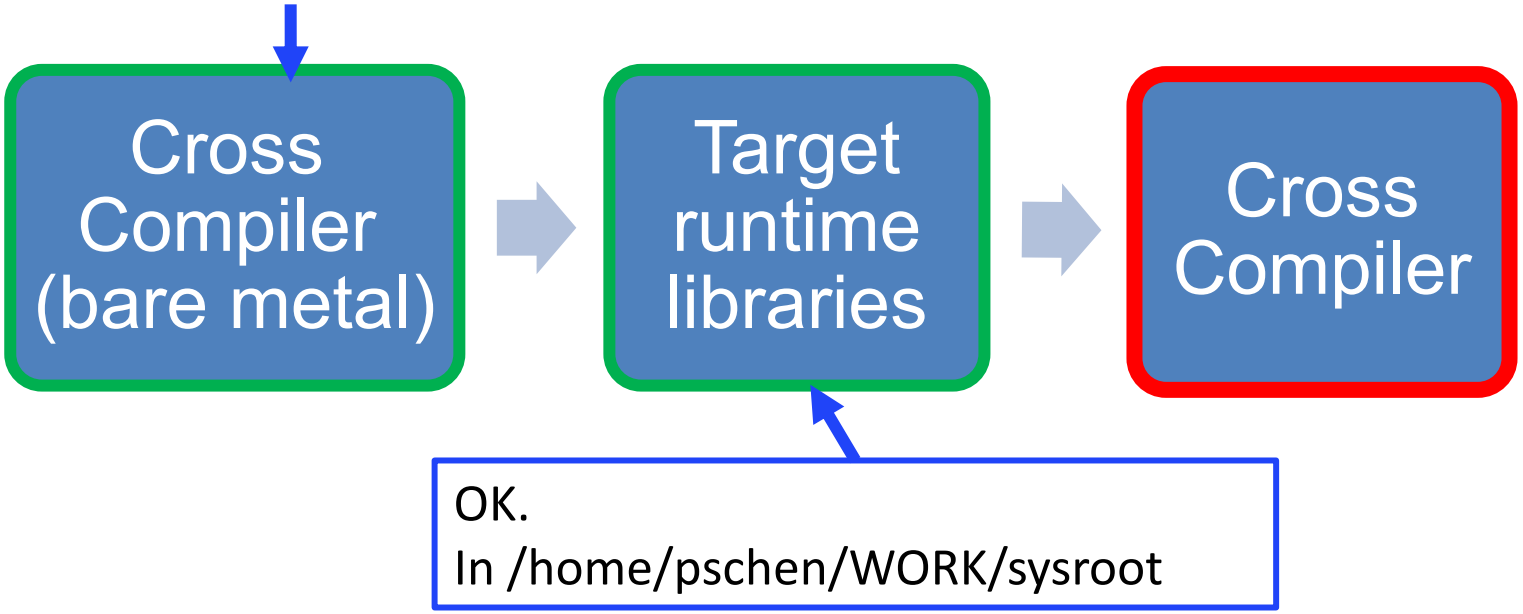
3-2. Building GLIBC

- Add /home/pschen/WORK/crossgcc1/bin to the PATH.

- `$ cd WORK`
- `$ tar -zxvf glibc-2.38.tar.gz`
- `$ mkdir build_glibc`
- `$ cd build_glibc`
- `$ ../glibc-2.38/configure --prefix=/usr \`
  `--host=arm-linux-gnueabihf \`
  `--target=arm-linux-gnueabihf \`
  `--with-headers=/home/pschen/WORK/sysroot/usr/include \`
  `--includedir=/usr/include --enable-add-ons \`
  `-disable-multilib CXX=arm-linux-gnueabihf-g++`
- `$ make`
- `$ make install install_root=/home/pschen/WORK/sysroot`

Begin to Build a Complete Cross Compiler

OK.
In /home/pschen/WORK/crossgcc1



4-1. Build Cross Binutils

- Change to the directory WORK
- $ mkdir build_binutils2
- $ cd build_binutils2
- $ ../binutils-2.40/configure \
  --prefix=**/home/pschen/WORK/crossgcc2** \
  **--target=arm-linux-gnueabihf **
  --with-sysroot=/home/pschen/WORK/sysroot
- $ make
- $ make install

4-2. Build a Cross Compiler (1)

- Add /home/pschen/WORK/**crossgcc2**/bin to the PATH.

- Change to the directory WORK
- $ mkdir build_gcc2
- $ cd build_gcc2

4-2. Build a Cross Compiler (2)

- $ ../gcc-12.3.0/configure \
  --prefix=**/home/pschen/WORK/crossgcc2** \
  --target=arm-linux-gnueabihf \
  --enable-languages=c \
  --with-sysroot=/home/pschen/WORK/sysroot
  --with-arch=**armv6** --with-fpu=vfp --with-float=hard \
  --disable-libmudflap --enable-libgomp \
  --disable-libssp --enable-libquadmath \
  --enable-libquadmath-support \
  --disable-libsanitizer --enable-lto \
  --enable-threads=posix --enable-target-optspace \
  --with-linker-hash-style=gnu --disable-nls \
  --disable-multilib --enable-long-long
- $make
- $make install

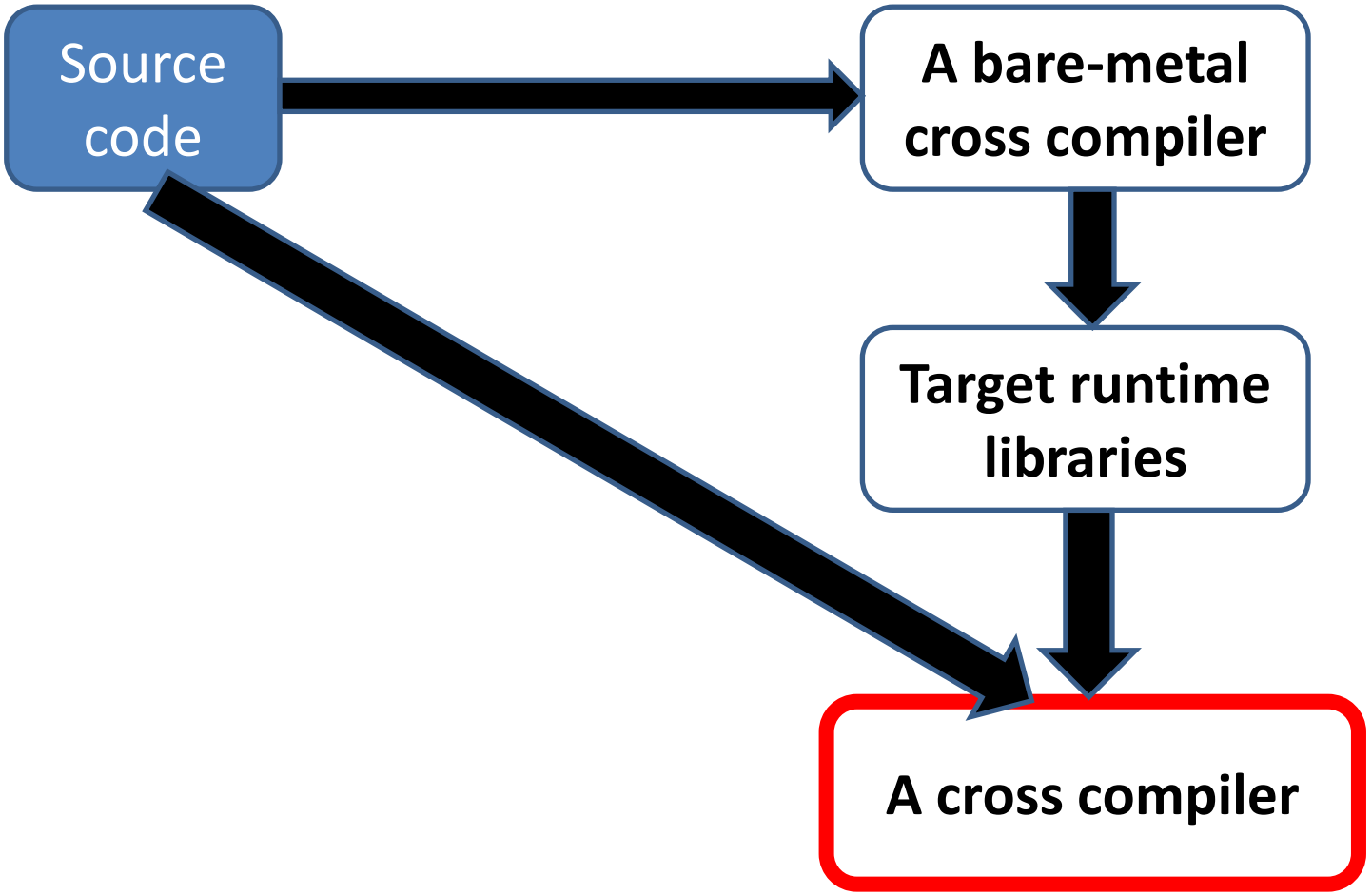
DEMO: Testing

- 驗證與測試
  - 若可編譯C program，並在ARM Linux上正確執行，則表示成功.
  - `$ /home/pschen/WORK/crossgcc2/bin/arm-linux-gnueabihf-gcc -S test.c`

    產生相對應的ARM組合語言test.s，這個測試只表示compiler可以產生ARM組合語言，但並不表示程式可以在ARM Linux上執行.

(測試的C程式請包含C的標準函式呼叫，以便驗證glibc有安裝成功)

Conclusion


Source code
A bare-metal cross compiler
Target runtime libraries
A cross compiler

Q&A (請於實驗報告裡回覆)

1. 請說明參數 “--with-sysroot”、”--with-arch“ 的目的為何?

2. 如果cross compiler是專門給Raspberry Pi 3B+使用，是否可以設定成”--with-arch=armv8”?